

HelixCode

HelixCode

作業を分割し、保存し、進行状況を決して失わない分散型AI開発プラットフォーム

概要

HelixCodeは、エンタープライズグレードのGoベース分散型AI開発プラットフォームです。SSHで管理されたワーカーのネットワークに開発作業を知的に分割し、自動チェックポイントとロールバック機能により作業の損失を防ぎます。複数のLLMプロバイダーとの統合、開発ライフサイクル全体のワークフロー、クロスプラットフォーム対応をREST、CLI、TUI、MCPインターフェースのもとに統合しています。

短い説明

HelixCodeは、Goで記述された分散型AI開発プラットフォームです。SSHベースのワーカーネットワークに作業を知的に分割し、自動チェックポイントとロールバックで進行状況を保護します。複数のLLMプロバイダーを統合し、REST、CLI、TUI、MCPインターフェースを通じて開発ライフサイクル全体をサポートします。

長い説明

HelixCodeは、エンタープライズグレードの分散型AI開発プラットフォーム（dev.helix.code、MITライセンス）であり、そのキャッチフレーズが示すシンプルな約束を文字通り実現します。それは「作業を分割し、保存し、進行状況を決して失わない」というものです。知的なタスク分割、自動作業保存、クロスプラットフォーム開発ワークフローを設計理念とし、分散コンピューティングに求められる並行処理と単一バイナリのポータビリティを実現するためにGoで記述されています。自動チェックポイント、ロールバック、リアルタイム監視は、オプションのアドオンではなく、第一級の機能として組み込まれています。

そのアーキテクチャは、REST + WebSocket + MCPのAPIレイヤーを、集中的なコアサービス群の上に構築しています。これには、JWT認証とセッション管理、ヘルスマonitoring付きSSHベースのワーカープール管理、チェックポイントと依存関係処理を備えたタスク管理、プロジェクトおよびワークフロー管理、統合LLMプロバイダーレイヤーが含まれます。これらはすべてPostgreSQL上に永続化され、Redisはオプションの調整およびキャッシング層として利用可能です。分散ワーカーはネットワーク上で自動インストールされるため、サーバーをマシンに向けるだけでスケールアウトが可能です。また、CLI、ターミナルUI、REST、モバイルフレームワークといった複数のクライアントインターフェースを備えているため、スクリプト、ターミナル、アプリから同じプラットフォームにアクセスできます。

HelixCodeは、開発ライフサイクル全体をエンドツーエンドでサポートします。計画、ビルド、テスト、リファクタリングのワークフローは、依存関係を認識し、複数セッションのコンテキストを追跡するため、長時間にわたる作業も中断やマシンの切り替えを跨いで継続できます。複数のLLMプロバイダー（llama.cpp、Ollama、OpenAI）を単一のインターフェースで統合し、利用可能なCPU/GPU/メモリを検出してモデルをマシンに適合させるハードウェア対応のモデル選択機能を備えています。また、単一の処理では不十分な問題に対して、連鎖思考やツリー思考といった高度な推論戦略もサポートしています。Model Context Protocolは複数のトランスポートを介して実装され、標準化されたツールやコンテキストの交換が可能です。さらに、Slack、Discord、メール、Telegramを通じたマルチチャネル通知により、分散作業の進捗状況をチームに伝えます。対応プラットフォームはLinux、macOS、Windows、Aurora OS、SymphonyOSです。

コンテンツ

なぜ開発したのか

分散型かつAI支援の開発では、タスクが複数のマシンに分割されたり中断されたりすると、コンテキストや進捗が失われがちです。HelixCodeは、タスクの分割をインテリジェントに行い、作業の保存を自動化することで、大規模な開発作業を分割し、ワーカーネットワークに分散させ、チェックポイントを設定し、状態を失うことなく再開やロールバックを可能にします。

なぜゲームチェンジャーなのか

分散型AI開発を「耐久性のある」ものに変える——これは、チームが手作業でこれらの要素を組み合わせていた時代には実現不可能だった機能です。従来は3つの異なるツールに分かれていたものが、1つのプラットフォームに統合されます。分散コンピューティング（SSHワーカーネットワークによる自動インストールとヘルスマonitoring）、AI開発支援（複数プロバイダーのLLMによる推論とツール呼び出し）、そしてフルライフサイクルのワークフロー自動化です。これらを結びつけるのは、データ

ベースを基盤としたチェックポイント機能です。タスクの状態、チェックポイント、依存関係がPostgreSQLに永続化されるため、複数のマシンやセッションにまたがるジョブでも、中断したまさにその地点からロールバックや再開が可能です。中断や作業の分割は、進捗の喪失要因ではなく、日常的で回復可能なイベントへと変わります。

革新的なポイント

- 作業保存をコア機能として実装：分散型開発タスクに自動チェックポイントとロールバックを適用し、中断やマシン障害が発生しても進捗が失われることなく、そのまま維持されます。
- ハードウェア認識型モデル選択：検出されたCPU/GPU/メモリを分析し、各タスクに対して実際に適切に動作するモデルを自動的に割り当て——ワーカーごとの手動調整は不要です。
- 1つのプラットフォーム、5つのアクセス方法：REST、WebSocket、CLI、TUI、そしてMCPを提供。MCP自体も複数のトランスポート経由で公開されており、ツールやエージェントはどのような接続方法でも統合可能です。
- クロスプラットフォーム対応：従来のデスクトップOS（Linux、macOS、Windows）にとどまらず、Aurora OSやSymphonyOSにも対応。多くのツールが無視するプラットフォームまでワーカーフリートを拡大します。

最大の技術的課題とその解決策

- 分散型で中断可能なタスクにおける作業の喪失防止。ジョブが複数のマシンに分割されると、クラッシュや中断が発生した場合、進行中の作業が孤立してしまうのが一般的です。私たちはタスク自体をチェックポイントと依存関係の担い手としてモデル化し、PostgreSQLに永続化することで、システムは最後の正常な状態にロールバックしたり、そこから再開したりできるようにしました。この耐久性は、脆弱なメモリ内状態ではなく、データ層に組み込まれています。
- 異種混合ワーカーフリートの管理。Linux、macOS、Windows、Aurora、SymphonyOSなどのマシンで構成されるネットワークは、可用性やセットアップが常に変動します。これに対処するため、SSHベースの登録、新規ノードへの自動インストール、継続的なヘルスマモニタリングを行う専用のワーカープールサービスを導入。これにより、マシンの出入りがあってもフリートは常に把握・制御可能な状態を維持します。
- プロバイダーとハードウェアの多様性。LLMのバックエンドやそれを実行するマシンは、性能が大きく異なります。これを統一されたLLMプロバイダーインターフェースで隠蔽し、ハードウェア検出（CPU/GPU/メモリ）と組み合わせることで、インテリジェントなモデル選択を実現。呼び出し側がプロバイダーやハードウェアを意識することなく、適切なモデルが適切なマシンに割り当てられます。

テックスタック

- **Go (1.26+ 内部モジュール)** — ゴルーチンベースの並行処理と単一バイナリ出力が、分散ワーカーシステムに求められる要件に完璧に合致するため採用。オーケストレーションに必要な低コストな並列処理と、あらゆるノードに自動インストール可能な自己完結型バイナリを実現。コアサービスとCLI/serverバイナリを全て内包。
- **Gin (HTTPフレームワーク)** — 軽量かつ高速なRESTレイヤーを低オーバーヘッドで提供するため採用。全てのクライアントが利用する /api/v1 インターフェース（認証、ワーカー、タスク、プロジェクト）を提供。
- **PostgreSQL 15+ (pgx/v5経由)** — チェックポイントやロールバックに不可欠なトランザクショナルな永続性を確保するため、耐久性のあるシステム・オブ・レコードとして採用。11のテーブルから成る分散コンピューティングスキーマ（ユーザー、ワーカー、タスク、プロジェクト、セッション、llm_providers、通知）を保持し、作業の永続化を実現。
- **Redis 7+ (オプション、go-redis/v9)** — キャッシュや調整層としてオプションで利用可能。ホットパスの高速化を図りつつ、最小限のデプロイではPostgresのみでも稼働可能なよう、ハードな依存関係にはしない設計。
- **SSH** — ワーカーコントロールのトランスポートとして採用。既に広く普及し、セキュリティも確立されているため、ワーカー登録、自動インストール、リモートコマンド実行をプール全体で行う際、専用エージェントの事前デプロイが不要。
- **Model Context Protocol (MCP)** — 標準化されたツールやコンテキストの交換プロトコルとして採用。外部ツールやエージェントが単一のオープンプロトコルを介して統合可能。クライアントの接続環境に応じたマルチトランスポート対応を実装。
- **LLMプロバイダー (llama.cpp、Ollama、OpenAI)** — ローカルおよびホステッド推論を統一インターフェースでカバーするため採用。ハードウェアに応じたタスクルーティングが可能で、呼び出し側はローカルモデルとホステッドモデルの違いを意識する必要なし。

ステータスと正直な注意点

- ステータス：ベータ版。READMEでは「完全完了／全5フェーズ」と自己申告されているが、これはプロジェクト側の主張であり、第三者による検証は行われていないため、本ページではベータ版として扱う。
- 上記の詳細は全てリポジトリのREADMEに基づく。マーケティング表現（キャッチコピーなど）は編集上のものであり、ソースメトリクスではない。

優先度： Helixプライマリ。